BANKACILIK YÖNETİMİ ve MOBİL ENTEGRASYON SİSTEMİ RAPORU

(01.06.2025)

Proje Üyeleri:

- **Hüseyin Furkan KIZILOĞLU**
- **Yusuf EKER**
- **Muhammed ÖZDEMİR**

Doç. Dr. Murat KARAKUŞ

İÇERİK

- Genel Bilgiler
- Özet
- Gereksinimler
    - Fonksiyonel Gereksinimler
    - Fonksiyonel Olmayan Gereksinimler
    - Teknik Gereksinimler
    - Kısıtlamalar
- Konunun Önemi ve Özgün Değer
- Amaç ve Hedefler
- Kullanılan Teknolojiler
    - Frontend
    - Backend
    - Veri Tabanı
- Sistem Mimarisi
    - Frontend
    - Backend
    - Veri Tabanı
    - Venv Dosyası
- Stack Yapısı ve history.tsx İçinde Kullanımı

Genel Bilgiler

Yürütücü: Hüseyin Furkan KIZILOĞLU	
Diğer Üyeler: Yusuf EKER, Muhammed ÖZDEMİR	
Proje Başlığı: Bankacılık Yönetimi ve Mobil Entegrasyon Sistemi	
Danışmanın Adı Soyadı: Doç. Dr. Murat KARAKUŞ	

Gereksinimler

Bu bölümde geliştirilen mobil bankacılık uygulamasının gereksinimleri dört ana başlık altında sınıflandırılmıştır: fonksiyonel gereksinimler, fonksiyonel olmayan gereksinimler, teknik gereksinimler ve sistemin karşı karşıya olduğu kısıtlamalar. Her bir gereksinim projenin hedefleri doğrultusunda sistemin sahip olması gereken özellikleri, performans ölçütlerini, teknolojik gereksinimleri ve dışsal sınırlamaları ifade etmektedir.

Gereksinimlerin belirlenmesinde temel alınan kriterler: güvenli, erişilebilir ve kullanıcı dostu bir bankacılık deneyiminin sağlanmasıdır. Projenin sonunda yapılan analizle her gereksinimin karşılanıp karşılanmadığı tek tek değerlendirilmiştir. Gerekli açıklamalar ilgili maddenin karşısına not edilmiştir.

1. **Fonksiyonel Gereksinimler**
    a. Güvenli işlemlerle çok kullanıcılı hesap yönetimini desteklemeli ✓
    b. Dağıtık sistemler arasında para yatırma, çekme ve transfer işlemlerini gerçek zamanlı olarak gerçekleştirebilmeli ✓
    c. Bankacılık işlemleri için mobil uygulama arayüzü sunmalı ✓

2. **Fonksiyonel Olmayan Gereksinimler**
    a. Dağıtık işlem süreçlerinde ACID uyumluluğu ✓
    b. Yüksek erişilebilirlik ve düşük kesinti süresi ✓
    c. Uçtan uca şifreleme ile güvenlik ✓

3. **Teknik Gereksinimler**
    a. Çok iş parçacıklı backend servisleri ve mesaj kuyruğu sistemlerinin kullanımı ✓
    b. İlişkisel veri tabanı kullanımı (Sqlite kullanıldı) ✓
    c. Mobil API entegrasyonu (REST/GraphQL) ✓

4. **Kısıtlamalar**
    a. Mevzuat uyumu (PCI DSS, GDPR)
    b. Yoğun saatlerde yüksek işlem hacmini desteklemek zorundadır ✓

Konunun Önemi ve Özgün Değer

Konunun Önemi

Dijital dönüşüm süreci bankacılık sektöründe kullanıcıların şubelere gitmeden işlem yapabilmesini mümkün kılan mobil uygulamaların geliştirilmesini zorunlu hâle getirmiştir. Günümüzde bireylerin finansal işlemlerini mobil cihazlar üzerinden güvenli, hızlı ve kesintisiz biçimde gerçekleştirebilmesi; bankacılık hizmetlerinde kullanıcı deneyimi, güvenlik ve erişilebilirlik kriterlerini ön plana çıkarmaktadır.

Bu bağlamda geliştirilen mobil çözümler yalnızca kullanıcı odaklı değil, aynı zamanda mimari açıdan sürdürülebilir ve modern altyapılara dayanmalıdır. Özellikle son yıllarda artan veri güvenliği ihtiyacı, kullanıcı kimliklerinin korunması ve yüksek işlem hacmi altında dahi sistemin performans göstermesi gibi etkenler yazılım geliştiricilere daha esnek, taşınabilir ve güvenli sistemler tasarlama sorumluluğu yüklemektedir.

Bu projede geliştirilen sistem bu ihtiyaçlara cevap verebilecek şekilde, kullanıcı dostu bir mobil arayüz ve güvenli bir backend altyapısı ile bütüncül bir çözüm sunmaktadır. Projede tercih edilen modern yazılım bileşenleri; düşük kaynak tüketimiyle çalışan sistem mimarisi, kimlik doğrulama mekanizmaları ve mobil cihazlarla uyumlu arayüz tasarımı ile hem bireysel kullanıcıların ihtiyaçlarına cevap vermekte, hem de yazılım geliştirme pratiği açısından öğretici bir model sunmaktadır.

Özgün Değer

Bu proje mobil bankacılık alanında bireysel geliştiricilerin de kurumsal ölçekte çalışan bir prototip ortaya koyabileceğini gösteren işlevsel bir uygulamadır. Sistem kullanıcı kayıt işlemlerinden para transferine, işlem geçmişinden profil yönetimine kadar birçok temel bankacılık fonksiyonunu kapsayacak şekilde geliştirilmiştir.

Yazılımın arka ucunda FastAPI ile modern REST mimarisi kurulmuş, kullanıcı doğrulama süreçleri JSON Web Token (JWT) yöntemiyle gerçekleştirilmiş ve mobil istemciyle güvenli bağlantı kurulmuştur. Mobil tarafta React Native ile platform bağımsız bir uygulama geliştirilmiş, Expo ile kurulum ve test süreçleri kolaylaştırılmıştır. Bu yapı sayesinde sistem hem geliştirici hem de son kullanıcı açısından sade, hızlı ve güvenli bir deneyim sunmaktadır.

Ayrıca uygulama içinde işlem geçmişi görüntüleme, IBAN ve e-posta ile transfer alternatifleri gibi detaylar yazılım mimarisi açısından işlevsel çeşitlilik kazandırmaktadır. Bu yönleriyle sistem sadece işlevsel bir uygulama değil, aynı zamanda yazılım mühendisliği eğitiminde referans gösterilebilecek düzeyde bir örnek çalışma niteliği taşımaktadır.

Amaç ve Hedefler

Amaçlar

Bu projenin temel amacı bireysel kullanıcıların temel bankacılık işlemlerini mobil cihazlar üzerinden güvenli, hızlı ve kullanıcı dostu bir şekilde gerçekleştirebilecekleri bir mobil bankacılık uygulaması prototipi geliştirmektir. Proje kapsamında kullanıcıların kaydolması, giriş yapması, bakiye

görüntülemesi, para yatırması, para çekmesi, para transferi yapması ve işlem geçmişini görüntülemesi gibi temel işlemlerinin sağlıklı biçimde gerçekleştirilmesi hedeflenmiştir.

Bu amaç doğrultusunda modern yazılım teknolojileri kullanılarak istemci-sunucu (client-server) mimarisi kurulmuş; mobil uygulama tarafında kullanıcı deneyimi ön planda tutulurken backend sistemde güvenlik, veri bütünlüğü ve API yapısı temel alınmıştır.

Hedefler

Bu projenin gerçekleştirilmesi sürecinde bir dizi somut ve ölçülebilir hedef belirlenmiştir. İlk olarak kullanıcıların sisteme güvenli bir şekilde kaydolabilmeleri ve giriş yapabilmeleri amacıyla JWT (JSON Web Token) tabanlı bir kimlik doğrulama mekanizması kurulmuştur. Sistem kullanıcıların mevcut bakiyelerini görüntüleyebilecekleri ve bu bilgilere sunucu üzerinden gerçek zamanlı olarak erişebilecekleri şekilde tasarlanmıştır. Ayrıca kullanıcıların uygulama üzerinden para yatırma, çekme ve transfer işlemlerini gerçekleştirebilmeleri için gerekli olan API servisleri oluşturulmuştur.

Bu bağlamda sistemde REST API yapısı tercih edilmiştir. REST API, HTTP protokolü üzerinden çalışan ve sistem kaynaklarına erişim sağlayan bir mimari tasarım yaklaşımıdır. Bu yapı veri üzerinde oluşturma (POST), okuma (GET), güncelleme (PUT/PATCH) ve silme (DELETE) gibi CRUD işlemlerini gerçekleştirmek için standart HTTP metotlarını kullanır. Bu projede RESTful servislerin hızlı, sade ve asenkron bir şekilde geliştirilmesini mümkün kılan modern bir Python web çatısı olan FastAPI tercih edilmiştir.

Transfer işlemleri için hem IBAN hem de e-posta kullanılarak para gönderimi yapılabilecek esnek bir yapı kurgulanmıştır. Bunun yanında kullanıcıların geçmişte gerçekleştirdikleri işlemleri görüntüleyebilecekleri bir işlem geçmişi ekranı da geliştirilmiştir. Uygulama hem Android hem de iOS cihazlarla uyumlu şekilde çalışacak biçimde geliştirilmiş, mobil kullanıcı deneyimi (UX) ilkeleri gözetilerek sade ve işlevsel bir arayüz sunulmuştur. Son olarak hem frontend hem de backend tarafında modüler bir yapı tercih edilerek sistemin sürdürülebilirliği ve ileride yapılacak genişletmeler için altyapı hazır hâle getirilmiştir.

Kullanılan Teknolojiler

Bu projede hem istemci (mobil uygulama) hem de sunucu (API sunucusu) tarafında modern ve yaygın kullanılan yazılım teknolojileri tercih edilmiştir. Kullanılan teknolojiler aşağıda sistem bileşenlerine göre kategorize edilerek açıklanmıştır:

1. Frontend (Mobil Uygulama)

Mobil uygulama tarafında platformlar arası (Android/iOS) geliştirme imkânı sunan React Native kullanılmıştır. Proje yapısı Expo çatısı altında organize edilmiş, ekran bileşenleri auth klasörü altında modüler biçimde yapılandırılmıştır.

- **React Native:** Mobil cihazlar için JavaScript/TypeScript tabanlı kullanıcı arayüzleri geliştirmeye olanak sağlar.
- **Expo:** React Native projelerinin kurulumu, test edilmesi ve dağıtımı için kullanılan araç takımıdır.
- **React Navigation / Expo Router:** Sayfalar arası geçişlerde yönlendirme işlemleri için tercih edilmiştir.
- **React Native Paper:** UI bileşenlerinin (butonlar, giriş kutuları, formlar vb.) modern ve sade şekilde tasarlanmasında kullanılmıştır.
- **AsyncStorage:** JWT token gibi kullanıcı oturum bilgilerini yerel olarak saklamak amacıyla kullanılmıştır.

2. Backend

Backend sistemi Python dili kullanılarak FastAPI çatısı ile geliştirilmiştir. Tüm kullanıcı ve işlem fonksiyonları REST API mimarisi üzerinden gerçekleştirilmiştir.

- **FastAPI:** RESTful servisleri hızlı ve asenkron olarak geliştirmeye imkân tanıyan modern Python frameworküdür.
- **Uvicorn:** FastAPI uygulamasının çalıştırılması için kullanılan ASGI uyumlu sunucudur.
- **SQLAlchemy:** Pythonda nesne tabanlı veri tabanı yönetimi sağlayan ORM kütüphanesidir. User ve Transaction modelleri bu yapı ile tanımlanmıştır.
- **JWT:** Kimlik doğrulama işlemleri için kullanılan token temelli güvenlik yöntemidir.
- **Passlib (bcrypt):** Kullanıcı şifrelerinin güvenli şekilde şifrelenmesi için kullanılmıştır.
- **Pydantic:** API giriş-çıkış verilerinin doğrulanması ve tip kontrolü için şema bazlı sistem sunar.

3. Veri Tabanı

- **SQLite:** Küçük ve orta ölçekli projelerde tercih edilen gömülü veri tabanı sistemidir. Proje içerisinde bank.db dosyası ile kullanılmaktadır. Bu projede taşınabilirliği ve kurulumu kolay olduğu için tercih edilmiştir. İleride SQLite yerine MySQL, PostgreSQL gibi daha büyük ölçekli sistemlere geçiş yapılabilecek esnekliktedir.

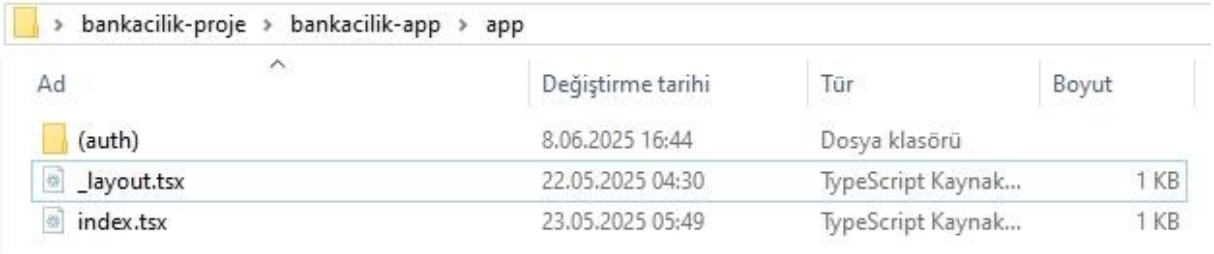
Sistem Mimarisi

Bu projede istemci (mobil uygulama) ile sunucu arasındaki iletişimi esas alan istemci-sunucu (client-server) temelli bir mimari uygulanmıştır. Uygulamanın temel bileşenleri üç ana yapıda organize edilmiştir: frontend (mobil arayüz), backend ve veri tabanı. Her bileşen kendi sorumluluk alanı içinde çalışmakta olup modüler yapı sayesinde sistem kolayca anlaşılabilir ve genişletilebilir hâle getirilmiştir.

1. Frontend (Mobil Uygulama Katmanı)

Mobil uygulama React Native teknolojisi ile geliştirilmiş ve Expo çatısı altında çalıştırılmıştır. Uygulama içerisindeki her işlevsel ekran (giriş, kayıt, transfer, işlem geçmişi vb.), auth klasöründe yer alan .tsx (TSX, TypeScript ile yazılmış ve JSX sözdizimi içeren dosyaların uzantısıdır. React gibi kütüphanelerle birlikte kullanılır.) dosyaları şeklinde modüler olarak yapılandırılmıştır. Arayüz ile sunucu arasındaki iletişim axios kütüphanesi üzerinden REST API çağrıları aracılığıyla gerçekleştirilmiştir. Kullanıcı oturumu süresince kimlik doğrulama için alınan JWT tokenlar cihaz üzerinde AsyncStorage kullanılarak saklanmıştır.

app klasörü, Expo Router mimarisine göre yapılandırılmıştır ve uygulamanın yönlendirme (routing) sisteminin merkezidir. Buradaki her .tsx dosyası veya klasör bir ekran anlamına gelir. Kullanıcı arayüzünün sayfaları burada tanımlanır.



Şekil 1: app klasörü yapısı

index.tsx uygulama açıldığında kullanıcıyı otomatik olarak /login sayfasına yönlendirir, _layout.tsx ise tüm sayfalarda ortak olarak çalışan bir yapı olup sayfa geçişlerinin düzgün çalışmasını ve genel UI düzenini sağlar.

(auth) klasörü, kullanıcı kimlik doğrulama ve finansal işlemlerle ilgili tüm ekranları içerir. Her .tsx dosyası bir sayfayı temsil eder ve doğrudan backend API'leriyle haberleşir. (auth) klasörünün içindeki dosyaların işlevleri şu şekildedir:

- login.tsx: Bu dosya kullanıcıların giriş yapmasını sağlayan ekranı içerir. Kullanıcıdan e-posta ve şifre alır, bu bilgileri API'ye gönderir ve başarılı giriş durumunda kullanıcıyı ana sayfaya yönlendirir. Ayrıca kaydolma sayfasına yönlendirme linki de içerir.

Şekil 2: login.tsx görüntüsü

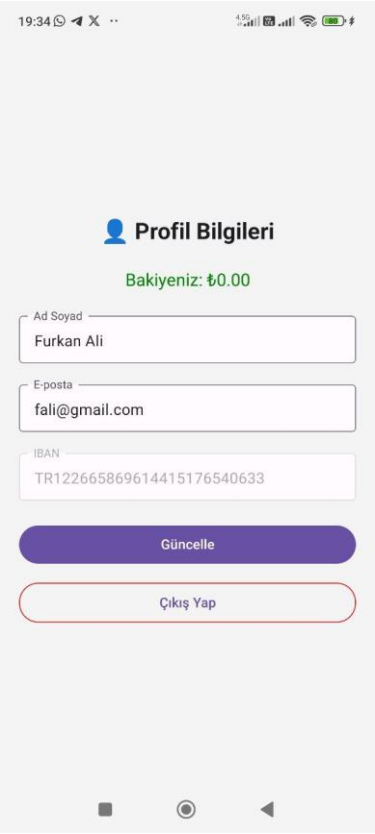
- register.tsx: Yeni kullanıcıların kaydolmasını sağlayan ekranı içerir. Kullanıcıdan ad soyad, e-posta ve şifre bilgilerini alır. Kayıt işlemi başarılı olduğunda kullanıcıya bilgi verir ve giriş sayfasına yönlendirme seçeneği sunar.

Şekil 3: register.tsx görüntüsü

- home.tsx: Kullanıcının ana sayfasını içerir. Burada kullanıcının bakiyesi görüntülenir ve çeşitli işlemler için butonlar bulunur:
    - Para gönderme
    - Para yatırma
    - Para çekme
    - İşlem geçmişi görüntüleme
    - Profil sayfasına gitme
    - Çıkış yapma

Şekil 4: home.tsx görüntüsü

- profile.tsx: Kullanıcının profil bilgilerini görüntüleme ve düzenleme sayfasıdır. Burada:
    - Kullanıcının ad soyadı
    - E-posta adresi
    - IBAN numarası
    - Güncel bakiye görüntülenir ve düzenlenebilir. Ayrıca çıkış yapma seçeneği de bulunur.



Şekil 5: profile.tsx görüntüsü

- deposit.tsx ve withdraw.tsx: Kullanıcının hesabına para yatırma ve çekme işlemini gerçekleştiren sayfalardır. Kullanıcıdan yatırmak veya çekmek istediği miktarı alır ve işlemi gerçekleştirir.

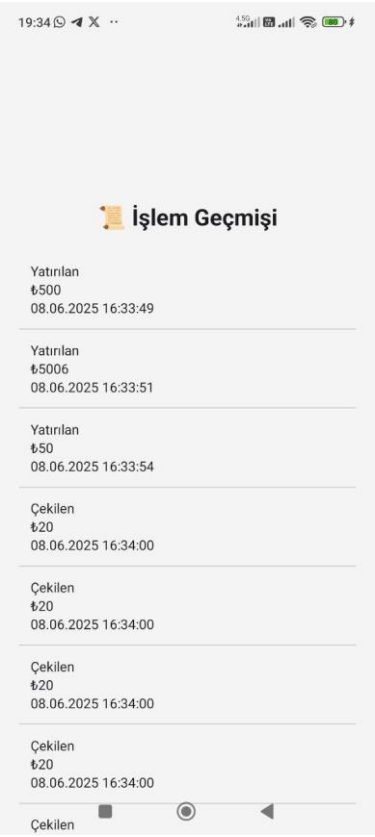
Şekil 6: deposit.tsx görüntüsü

Şekil 7: withdraw.tsx görüntüsü

- transfer.tsx: Bu dosya para transferi işlemlerini gerçekleştiren ekranı içerir. Önemli özellikleri:
    - Kullanıcılar iki farklı yöntemle para transferi yapabilir: E-posta adresiyle ve IBAN numarasıyla.
    - Kullanıcı arayüzünde transfer yöntemi seçimi için radio butonlar, alıcı bilgisi için giriş alanı (e-posta veya IBAN), transfer tutarı için sayısal giriş alanı, transfer butonu bulunur.

Şekil 8: transfer.tsx görüntüsü

- history.tsx: Bu dosya kullanıcının tüm işlem geçmişini görüntüleyen ekranı içerir. İşlem geçmişi verilerini yönetmek için özel bir Stack sınıfı kullanır. Geçmiş işlemlerin listelenmesi önce son işlemden başlaması gerektiği için (LIFO prensibi) Stack yapısıyla temsil edildi.



Şekil 9: history.tsx görüntüsü

2. Backend Katmanı

Backend FastAPI frameworkü kullanılarak Python dilinde geliştirilmiştir. Tüm servisler REST mimarisine uygun biçimde yapılandırılmış olup veri işleme ve doğrulama işlemleri Pydantic ile yapılmaktadır. Sunucu tarafında temel olarak şu yapılar yer almaktadır:

- main.py: Uygulamanın başlangıç dosyasıdır ve FastAPI sunucusunun ayağa kalkmasını sağlar.
- routes/: Kullanıcı işlemleri (users.py) ve finansal işlemler (transactions.py) bu klasörde yer alır.
    - users.py: Kullanıcı işlemlerini yöneten API uç noktalarını içerir. Kullanıcılar bu dosya sayesinde sisteme kaydolabilir, giriş yapabilir, bakiye bilgilerini görüntüleyebilir ve profil bilgilerini güncelleyebilirler. Kayıt işlemi sırasında kullanıcı için rastgele bir IBAN oluşturulur ve şifre güvenli bir şekilde şifrelenerek veri tabanına kaydedilir. Giriş işlemi sonucunda JWT tabanlı bir erişim tokenı döndürülür. Kullanıcı bilgileri yalnızca kimliği doğrulanmış kullanıcı tarafından görüntülenebilir ve yalnızca ad soyad ile e-posta bilgileri güncellenebilir, IBAN sabit kalır.
    - transactions.py: Para transferi işlemleriyle ilgili uç noktaları sağlar. Kullanıcılar başka bir kullanıcıya e-posta veya IBAN bilgisi üzerinden para transferi gerçekleştirebilir. Ayrıca sisteme para yatırabilir veya sistemden para çekebilirler. Bu işlemler sırasında kullanıcının bakiyesi güncellenir ve her hareket bir Transaction nesnesi olarak veri tabanına kaydedilir. İşlem geçmişi görüntülenmek istendiğinde kullanıcının tüm para hareketleri zaman sırasına göre listelenir ve işlemin türüne (Gönderilen, Alınan, Yatırılan, Çekilen) göre etiketlenir. Her işlem sırasında geçerli bakiye kontrolü yapılır ve yetersiz bakiye durumlarında kullanıcıya hata mesajı döndürülür. Bu yapı sistemin güvenli, takip edilebilir ve kullanıcı dostu bir şekilde çalışmasını sağlar.
- auth.py: JWT token üretimi ve şifre doğrulama işlemleri burada yürütülür. uygulamanın kimlik doğrulama sistemini yönetir. Kullanıcı şifrelerinin güvenli şekilde saklanması için hash_password ve verify_password işlevleri ile bcrypt algoritması kullanılır. Giriş yapan kullanıcılar için JWT (JSON Web Token) üretimi create_access_token fonksiyonu ile yapılır. Ayrıca get_current_user, gelen JWT tokenı çözerek aktif kullanıcıyı tespit eder. Bu yapı sayesinde kullanıcıya özel işlemler güvenli şekilde yapılabilir.
- models.py: Projenin veri tabanı ile Python kodları arasında köprü kuran veri modelleme dosyasıdır. Bu dosyada SQLAlchemy kütüphanesi kullanılarak User ve Transaction adında iki sınıf tanımlanmıştır.
- schemas.py: API ile haberleşmede kullanılan veri yapılarını tanımlar. Pydantic modeli kullanılarak UserCreate, UserLogin ve UserOut gibi sınıflar oluşturulmuştur. Bu sınıflar, gelen isteklerin doğruluğunu kontrol eder ve istemciye gönderilecek cevabın formatını belirler.
- database.py: Veri tabanı bağlantısı ve oturum yönetimi burada yapılandırılmıştır. uygulamanın veri tabanı ile olan bağlantısını ve oturum yönetimini sağlayan dosyadır. SQLite kullanılarak bank.db adlı dosya veri tabanı olarak tanımlanmıştır. engine, SessionLocal ve Base gibi SQLAlchemy bileşenleri burada oluşturulmuştur. Ayrıca, get_db() fonksiyonu sayesinde her istek için veri tabanı oturumu açılır ve güvenli şekilde kapatılır. Bu, bağımlılık enjeksiyonu yoluyla diğer modüllerde kullanılarak veri tabanı erişimini kolaylaştırır ve kod tekrarını azaltır.

3. Veri Tabanı Katmanı

Projede veri yönetimi için SQLite kullanılmıştır. Tüm kullanıcı ve işlem verileri bank.db adlı dosyada saklanmaktadır. SQLite gömülü bir veri tabanı olduğu için projenin taşınabilirliğini artırmış, geliştirmenin test ve prototipleme aşamalarında hafif ve pratik bir çözüm sunmuştur. Veri tabanına erişim SQLAlchemy ORM aracılığıyla yapılmıştır.

Bank.db veri tabanı users ve transactions tablolarından oluşur. users tablosu kullanıcılara ait kimlik ve hesap bilgilerini tutar. Transactions tablosu ise kullanıcılar arasında gerçekleşen para transferi işlemlerini ve para yatırma-çekme işlemlerini kayıt altına alır.

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated
1	id	INTEGER							
2	full_name	TEXT							
3	email	VARCHAR							
4	iban	TEXT							
5	hashed_password	VARCHAR							
6	balance	FLOAT							

Şekil 10: users tablosu yapısı

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated
1	id	INTEGER							
2	sender_id	INTEGER							
3	receiver_id	INTEGER							
4	amount	FLOAT							
5	timestamp	DATETIME							

Şekil 11: transactions tablosu yapısı

	id	full_name	email	iban	hashed_password	balance
1	1	NULL	alice@mail.com	NULL	$2b$12$51uOP1LjkLNZqRj/JnJWGuFySl6VDsWBnad2VTW4Edv0E3zuG24gC	100
2	2	NULL	bob@mail.com	NULL	$2b$12$R9HKoeciquyH3pZvnDWPvucf6.naDAHiXRlcR4zDDCL8FsyTkt3ym	183834
3	3	FK	furkan@mail.com	TR43	$2b$12$TjRrxAqtk.m.33TTNvr9PuxdB/ijSsmbwHoBZqkSqosU/d88Gymtu	78756.4
4	4	NULL	furkan1@mail.com	NULL	$2b$12$Gu5ESLxEWWpOg6BoPszLCOD3Jh9j0HfaWhwZPopQP577KUM/Nh2wK	2036
5	5	NULL	assjsk	NULL	$2b$12$cTHTcB0puP9TDeeMxvDkW.SU6VJlMZWdGc.Rt0yykzORcLQhrRI6y	0
6	6	NULL	Yeker277@gmail.com	NULL	$2b$12$MxU2BjI3wph6KuKFJCApA.l9jv3P4QUW0younyj6lai/nXPQr2VLK	120901
7	7	furkan	furkan12@mail.com	TR6317102946361999921712 23	$2b$12$tN/ae668zAw2LyMBY/jDnueqmM/EQsEJEuy04jHW5v87SsmC6SM9.	3208.199999999999
8	8	Furkan Y	furkanr	TR5388723906368396789937 10	$2b$12$YO2LnyAOsSadmjKQU/jQN.KUODUCFuHYggavxiib1k0s7cn9rnWdi	170
9	9	string	user@example.com	TR0002532933452364542672 08	$2b$12$VAuRAzS4kg0bnJhPn0uptOsRMQzBvzcGbjNasItIwK/TjkeTOEKf2	0
10	10	ali	ali@gmail.com	TR2273389787972063730293 65	$2b$12$P6NutLAxTsyE7I/JmBa/8.gGWJ6aCSdS7XZkzgyA.xeEGRfag9tr.	1600000
11	11	Furkan Kızıloğlu	huseyinfurkankiziloglu1@gmail.com	TR9976048605021327707547 61	$2b$12$Xn3oYE3R93lpx7mDOow2/Om/uwIA5SKPxKXoX8jiwlRHGnAfcvuaO	2000
12	12	Ahmet Ayten	ahmetayten@gmail.com	TR2707405507965716068263 69	$2b$12$hQHOcWux8xatZYlqeIpaAODI.IIfm3iJcMVwVeIhzWjevc32mtQTG	0
13	13	Aslan Amca	aslanamca@gmail.com	TR6159651585756019522505 26	$2b$12$claDBrJXJks4vbFJ/N2cpOMhBE9c4B2t9piHAZo6K1/ifuupMk8Hq	0

Şekil 12: users tablosu içeriği

	id	sender_id	receiver_id	amount	timestamp
1	281	12	11	2000	2025-06-08 15:17:49.340674
2	280	12	NULL	500	2025-06-08 15:16:50.546527
3	279	12	NULL	500	2025-06-08 15:16:49.829334
4	278	12	NULL	500	2025-06-08 15:16:49.066493
5	277	NULL	12	500	2025-06-08 15:16:44.162977
6	276	NULL	12	1000	2025-06-08 15:16:41.633514
7	275	NULL	12	1000	2025-06-08 15:16:40.912822
8	274	NULL	12	1000	2025-06-08 15:16:40.082753
9	273	NULL	10	800000	2025-06-08 14:00:14.122182
10	272	NULL	10	800000	2025-06-08 14:00:11.470570
11	271	2	4	500	2025-05-22 20:07:07.434835
12	270	2	4	500	2025-05-22 20:07:06.812494
13	269	2	4	500	2025-05-22 20:07:06.181949
14	268	2	4	500	2025-05-22 20:07:05.190373
15	267	NULL	2	5000	2025-05-22 20:06:40.725525
16	266	4	6	85	2025-05-22 19:07:48.659847
17	265	4	8	85	2025-05-22 19:07:15.625474
18	264	4	8	85	2025-05-22 19:07:05.841984
19	263	7	4	32	2025-05-22 18:49:13.263903
20	262	7	4	31.8	2025-05-22 18:49:08.934031
21	261	7	4	31.8	2025-05-22 18:49:08.021476
22	260	7	4	31.8	2025-05-22 18:49:07.491235
23	259	7	4	31.8	2025-05-22 18:49:06.809504
24	258	7	4	31.8	2025-05-22 18:49:04.211841
25	257	7	4	50	2025-05-22 18:48:46.625124
26	256	7	4	50	2025-05-22 18:48:44.771249
27	255	7	7	50	2025-05-22 18:48:24.625412
28	254	7	NULL	500	2025-05-22 18:47:50.268485
29	253	7	NULL	500	2025-05-22 18:47:49.346623
30	252	7	NULL	500.8	2025-05-22 18:47:46.539892
31	251	NULL	7	5000	2025-05-22 18:47:30.887299

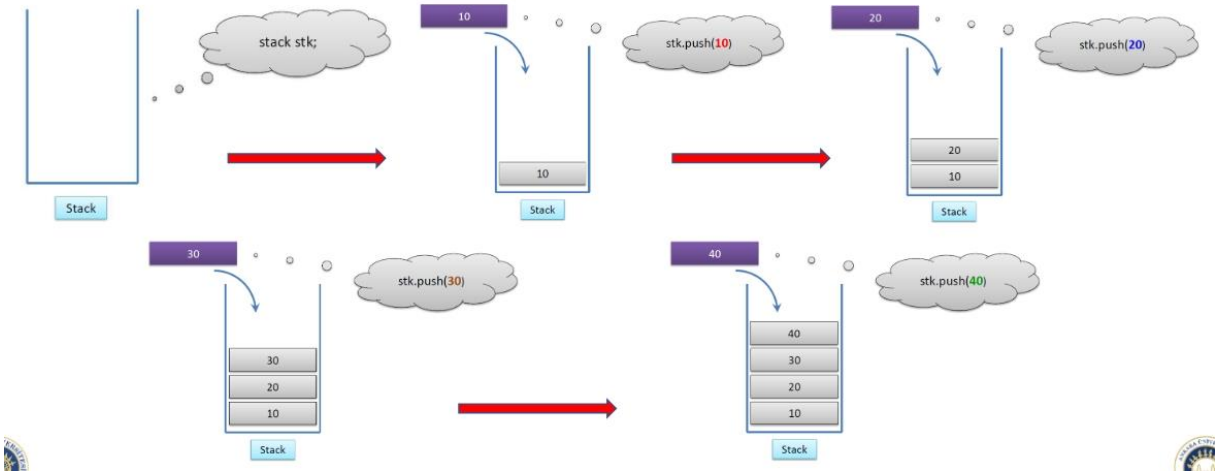
Şekil 13: transactions tablosu içeriği

4. Venv Dosyası

Venv klasörü, projenin kök dizininde yer alıyor ve bu projeye ait Python sanal ortamını barındırıyor. Yani bu klasör bu projede kullanılan Python kütüphanelerini içerir (örneğin: fastapi, sqlalchemy, passlib, uvicorn vb.).

STACK YAPISI VE history.tsx İÇİNDE KULLANIMI

Stack (Yığın) veri yapılarında son giren elemanın ilk çıkarıldığı bir yapıdır. Bu prensibe LIFO (Last In, First Out – Son Giren İlk Çıkar) denir.



Şekil 14: Doç. Dr. Murat Karakuş ders notları – Stack yapısı

Bu proje kapsamında işlem geçmişi verilerini sondan başa doğru (en yeni işlem en önce görünsün) sıralamak için bir Stack sınıfı tanımlanmıştır. Kodda bu yapı sayesinde işlemler LIFO sırasına çevrilmiştir.

```
class Stack<T> {
  private items: T[] = [];

  push(item: T) {
    this.items.push(item);
  }

  pop(): T | undefined {
    return this.items.pop();
  }

  peek(): T | undefined {
    return this.items[this.items.length - 1];
  }

  toArray(): T[] {
    return [...this.items].reverse(); // LIFO
  }

  isEmpty(): boolean {
    return this.items.length === 0;
  }
```

```
}
```

Şekil 15: Stack sınıfı tanımı (history.tsx içinde)

```
// Gelen veriyi stack'e push et
      const txStack = new Stack<any>();
      response.data.forEach((tx: any) => {
        txStack.push(tx);
      });

      setTransactions(txStack.toArray());
```

Şekil 16: Verilerin Stack’e eklenmesi (history.tsx içinde)

İşlem geçmişi en yeni işlem en üstte olacak şekilde sıralanır. Bu yüzden projede işlem geçmişi gösterilirken Stack yapısı tercih edildi.